

Critical Thinking Module 2: Process Management

Alexander Ricciardi

Colorado State University Global

CSC507: Foundations of OSs

Professor: Dr. Joseph Issa

May 24, 2026

Critical Thinking Module 2: Process Management

Process management is one of the most important responsibilities of an Operating System (OS) because programs do not simply run on their own. The OS must decide which process receives CPU time, track process state, coordinate memory use, and provide controlled ways to inspect or stop active work. Bazuku et al. (2023) describe the OS as the layer that coordinates hardware and software, while Rawat (2025) identifies process management as one of the central functions through which that coordination takes place.

This paper compares the process-management utilities of Windows and Ubuntu/Linux by performing six related tasks on each platform. The tasks were viewing running processes, monitoring CPU and memory usage, finding a specific process and its Process ID (PID), terminating a process, changing process priority, and locating the executable file behind a running process. The Windows evidence came from Task Manager, while the Linux evidence came from shell commands including ``ps``, ``top``, ``pgrep``, ``ps -p``, ``renice``, ``readlink``, and ``kill`` (Fisher, n.d.; Kerrisk, n.d.-a, n.d.-b, n.d.-c, n.d.-d, n.d.-e, n.d.-f; Microsoft Learn, 2026; Microsoft Support, n.d.). It is important to note that the screenshots came from two different systems, Windows 11 and Ubuntu 24.04 LTS, so the goal of this paper is not to compare raw hardware performance. Instead, the focus is on how each OS exposes process information, how easily the user can intervene, and how clearly each utility reveals the underlying logic of process control.

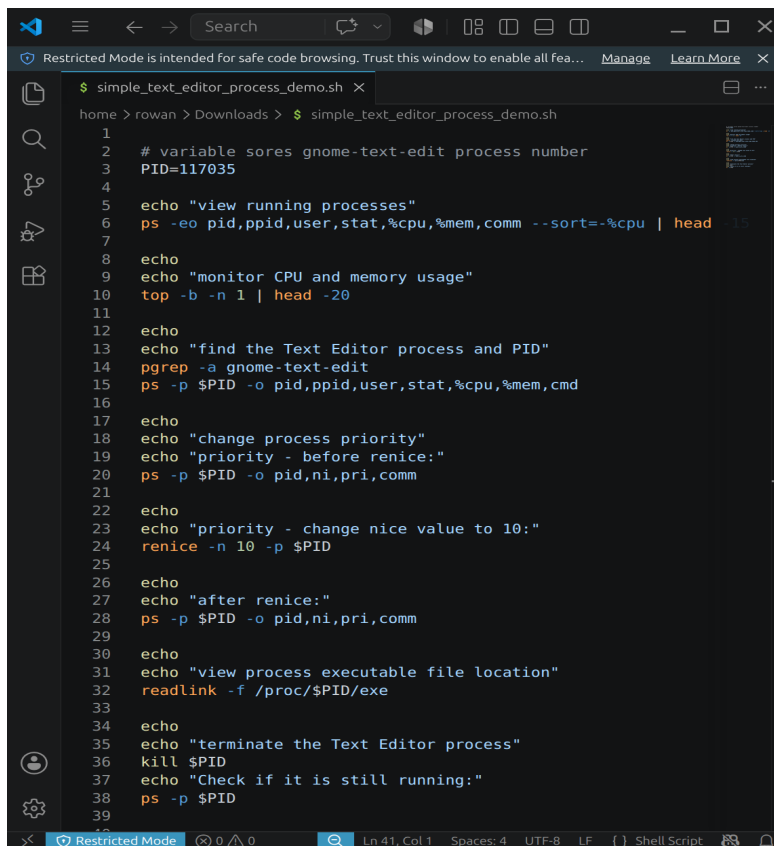
Process Management Utilities Used

Windows used Task Manager, especially the Processes, Performance, and Details tabs; see Figures 3, 4, and 5. Those views made it possible to see running applications, monitor CPU and memory activity, inspect process IDs, end tasks, adjust process priority, and open the file location of a selected executable. Microsoft describes Task Manager as a tool for troubleshooting

processes and resource usage, and the captured screenshots support that description by showing both summary-level and process-level information (Microsoft Learn, 2026; Microsoft Support, n.d.). Ubuntu/Linux used a sequence of shell commands against a `gnome-text-editor` test process; see Figures 1, 2, and 6. The command sequence included `ps -eo pid,ppid,user,stat,%cpu,%mem,comm --sort=-%cpu | head -15`, `top -b -n 1 | head -20`, `pgrep -a gnome-text-edit`, `ps -p \$PID -o ...`, `renice -n 10 -p \$PID`, `readlink -f /proc/\$PID/exe`, and `kill \$PID`. These commands covered observation, diagnosis, scheduling adjustment, executable-path lookup, and termination in a compact but technically meaningful workflow.

Figure 1

Visual Studio Code view of the Ubuntu shell script



```

$ simple_text_editor_process_demo.sh
home > rowan > Downloads > $ simple_text_editor_process_demo.sh
1
2 # variable sores gnome-text-edit process number
3 PID=117035
4
5 echo "view running processes"
6 ps -eo pid,ppid,user,stat,%cpu,%mem,comm --sort=-%cpu | head -15
7
8 echo
9 echo "monitor CPU and memory usage"
10 top -b -n 1 | head -20
11
12 echo
13 echo "find the Text Editor process and PID"
14 pgrep -a gnome-text-edit
15 ps -p $PID -o pid,ppid,user,stat,%cpu,%mem,cmd
16
17 echo
18 echo "change process priority"
19 echo "priority - before renice:"
20 ps -p $PID -o pid,ni,pri,comm
21
22 echo
23 echo "priority - change nice value to 10:"
24 renice -n 10 -p $PID
25
26 echo
27 echo "after renice:"
28 ps -p $PID -o pid,ni,pri,comm
29
30 echo
31 echo "view process executable file location"
32 readlink -f /proc/$PID/exe
33
34 echo
35 echo "terminate the Text Editor process"
36 kill $PID
37 echo "Check if it is still running:"
38 ps -p $PID
39

```

Note: The screenshot shows the Linux script, in the VS Code IDE, used to execute the process-management tasks in sequence.

Figure 2

Ubuntu terminal output for the Linux process-management tasks

```

rowan@ai01: ~
view running processes
  PID    PPID  USER      STAT %CPU %MEM COMMAND
 117472  112712 rowan      R+   100  0.0 ps
 113228  23546 rowan      Sl   15.0  0.3 chrome
 116056  115940 rowan      Sl    5.0  0.2 code
 46559   23546 rowan      Sl    4.9  0.6 chrome
 24759   23546 rowan      Sl    2.9  0.2 chrome
 113004  23546 rowan      Sl    2.1  0.1 chrome
 23572   23543 rowan      Sl    1.9  0.1 chrome
 116227  115937 rowan      Sl    1.8  0.0 code
 115937  2673   rowan      Slsl  1.0  0.1 code
 117358  2673   rowan      Sl    0.5  0.1 gnome-text-edit
 112699  2673   rowan      Ssl   0.4  0.0 gnome-terminal-
 23524   3287   rowan      Sll   0.4  0.3 chrome
 113091  23546 rowan      Sl    0.3  0.2 chrome
 117465  3827   rowan      SNl   0.3  0.0 tracker-extract

monitor CPU and memory usage
top - 17:52:52 up 6 days, 11:24, 1 user, load average: 0.47, 0.45, 0.49
Tasks: 548 total, 1 running, 547 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.6 us, 0.6 sy, 0.0 ni, 98.6 id, 0.3 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 128488.9 total, 110898.1 free, 9011.1 used, 11958.9 buff/cache
MiB Swap: 8192.0 total, 8192.0 free, 0.0 used, 119477.8 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
 2944 rowan     20   0  25.1g 259692 166416 S   9.1   0.2   6:59.55 Xorg
 23572 rowan     20   0  49.2g 261784 139120 S   9.1   0.2 178:34.47 chrome
 24759 rowan     20   0 1396.4g 270508 125908 S   9.1   0.2 277:14.66 chrome
 113004 rowan     20   0 1396.4g 261252 125876 S   9.1   0.2 1:00.39 chrome
 113091 rowan     20   0 1394.5g 271292 141168 S   9.1   0.2 0:09.80 chrome
 117474 rowan     20   0  23180    5688   3516 R   9.1   0.0 0:00.01 top
    1 root      20   0  23684   15064   9636 S   0.0   0.0 0:26.06 systemd
    2 root      20   0      0      0      0 S   0.0   0.0 0:00.17 kthreadd
    3 root      20   0      0      0      0 S   0.0   0.0 0:00.00 pool_wo+
    4 root      0 -20      0      0      0 I   0.0   0.0 0:00.00 kworker+
    5 root      0 -20      0      0      0 I   0.0   0.0 0:00.00 kworker+
    6 root      0 -20      0      0      0 I   0.0   0.0 0:00.00 kworker+
    7 root      0 -20      0      0      0 I   0.0   0.0 0:00.00 kworker+

find the Text Editor process and PID
117358 /usr/bin/gnome-text-editor /home/rowan/Downloads/simple_text_editor_process_demo.sh
  PID    PPID  USER      STAT %CPU %MEM CMD
 117358  2673   rowan      Sl    0.5  0.1 /usr/bin/gnome-text-editor

change process priority
priority - before renice:
  PID  NI PRI COMMAND
 117358  0 19 gnome-text-edit

priority - change nice value to 10:
117358 (process ID) old priority 0, new priority 10

after renice:
  PID  NI PRI COMMAND
 117358 10 9 gnome-text-edit

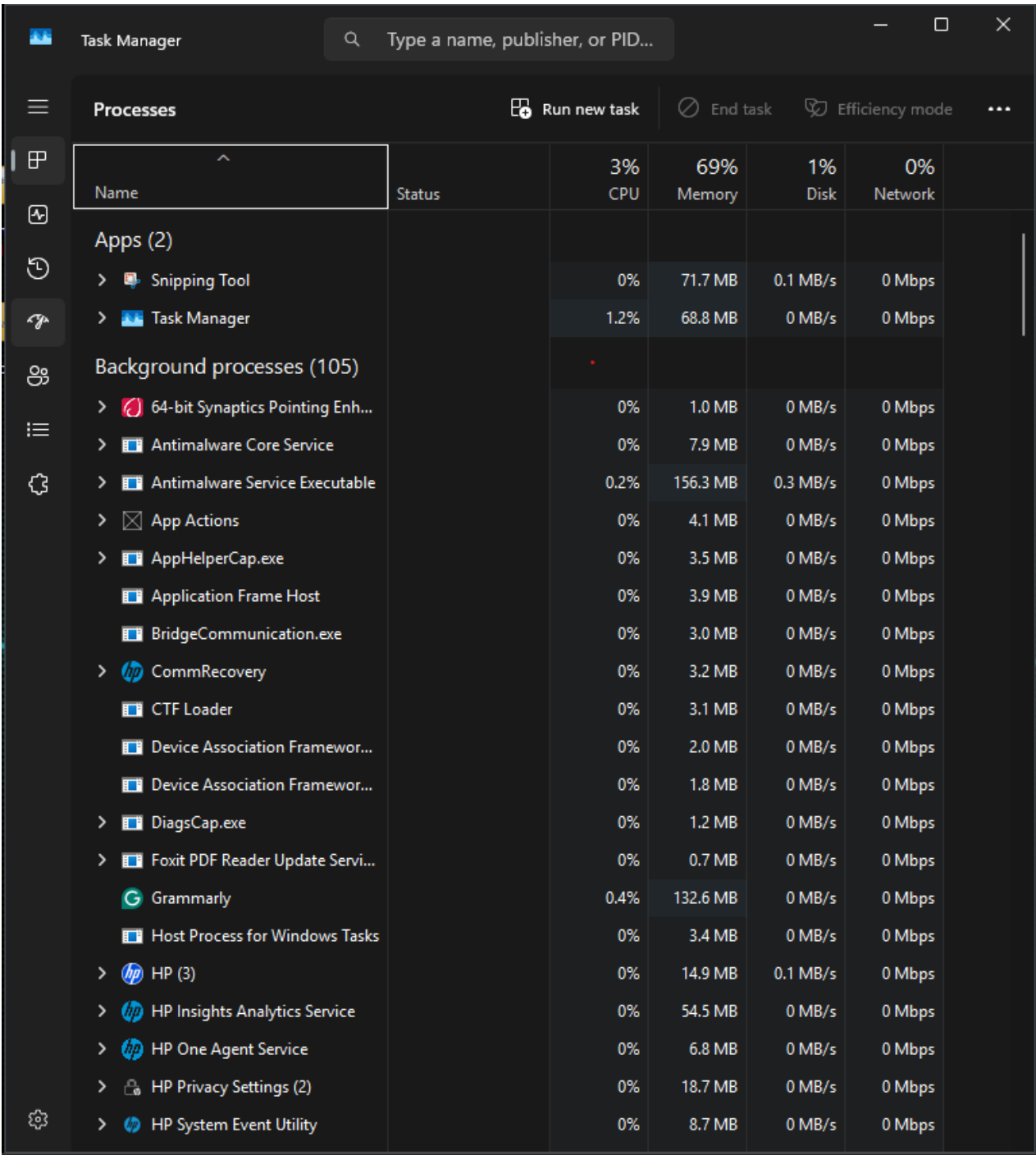
view process executable file location
/usr/bin/gnome-text-editor

terminate the Text Editor process
Check if it is still running:
  PID TTY          TIME CMD
rowan@ai01: ~$

```

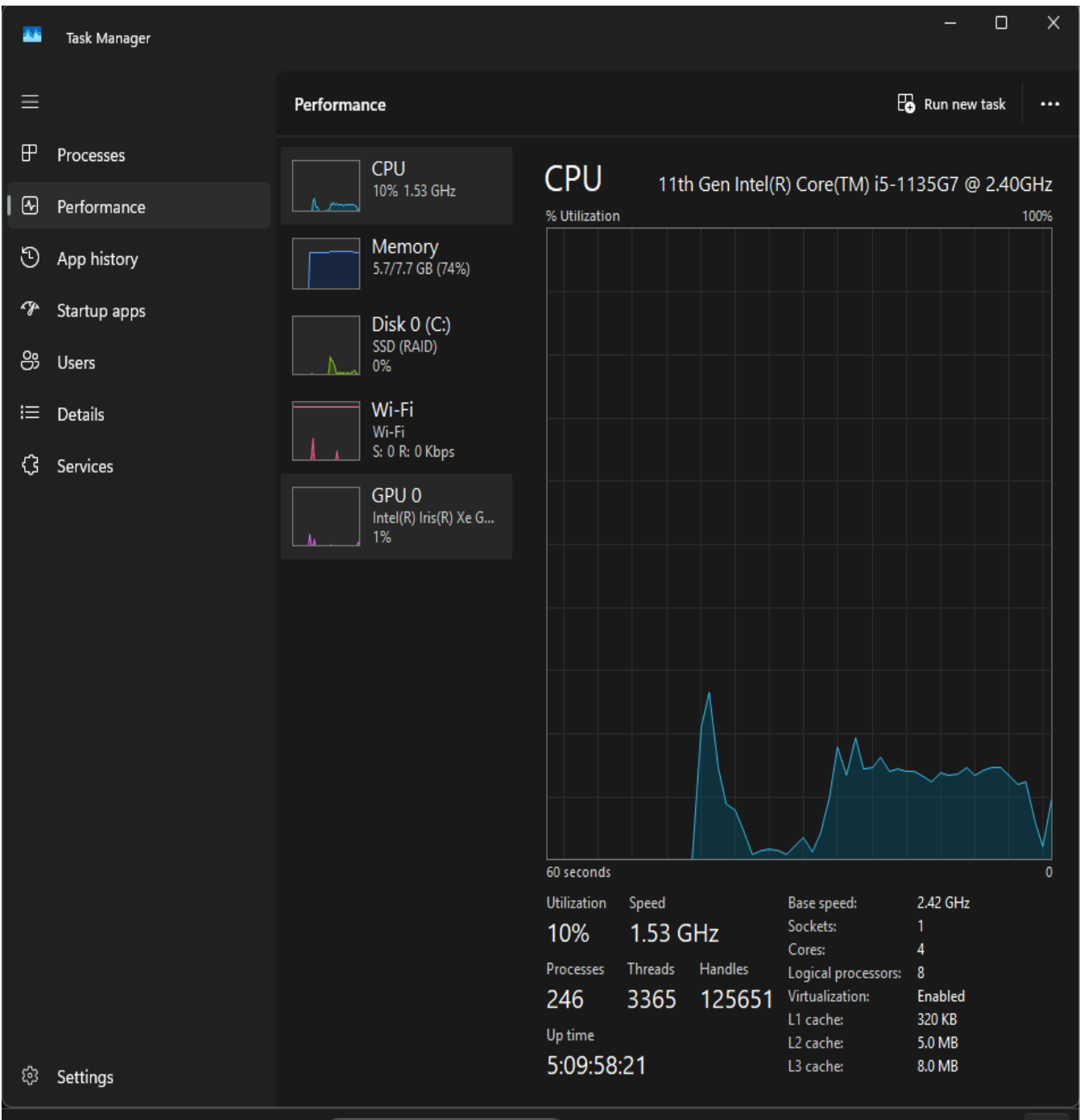
Note: The screenshot shows the Linux evidence used for process listing, resource monitoring, PID identification, priority change, executable-path lookup, and termination.

Figure 3
Windows Task Manager Processes view



Note: The screenshot shows Windows applications and supporting services grouped in the Processes tab.

Figure 4
Windows Task Manager Performance view



Note: The screenshot shows CPU utilization, memory use, and system-level execution data in the Performance tab.

Figure 5

Windows Task Manager Details view with PID column visible

Name	PID	Status	User name	CPU	Work...	Platform	UAC virtualizat...
AggregatorHost.exe	7308	Running	SYSTEM	00	0 K	64 bit	Not allowed
AppActions.exe	16632	Running	copya	00	-12 K	64 bit	Disabled
AppHelperCap.exe	3452	Running	SYSTEM	00	0 K	64 bit	Not allowed
ApplicationFrame...	2112	Running	copya	00	0 K	64 bit	Disabled
AppVShNotify.exe	1564	Running	SYSTEM	00	0 K	64 bit	Not allowed
backgroundTaskH...	8988	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	11696	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	16304	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	8700	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	3936	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	14840	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	11796	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	11252	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	18756	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	10612	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	12412	Suspended	copya	00	0 K	64 bit	Disabled
BridgeCommunica...	1960	Running	copya	00	0 K	64 bit	Disabled
conhost.exe	9924	Running	copya	00	0 K	64 bit	Disabled
CrossDeviceResum...	7076	Running	copya	00	-8 K	64 bit	Disabled
CrossDeviceService...	10704	Running	copya	00	0 K	64 bit	Disabled
csrss.exe	956	Running	SYSTEM	00	0 K	64 bit	Not allowed
csrss.exe	700	Running	SYSTEM	00	8 K	64 bit	Not allowed
ctfmon.exe	15156	Running	copya	00	8 K	64 bit	Disabled
dashHost.exe	7744	Running	LOCAL SE...	00	0 K	64 bit	Not allowed
dashHost.exe	6064	Running	NETWORK...	00	0 K	64 bit	Not allowed
DesktopExtension.e...	19072	Running	copya	00	4 K	64 bit	Disabled
DiagsCap.exe	3468	Running	SYSTEM	00	0 K	64 bit	Not allowed
dptf_helper.exe	8872	Running	copya	00	0 K	64 bit	Disabled
dwm.exe	1596	Running	DWM-1	02	-204 K	64 bit	Disabled

Note: The screenshot shows the Details tab, which exposes per-process identifiers and lower-level management controls.

Viewing Running Processes

Viewing the processes that are running is the basic task, but an essential process-management task, because it answers the question of what the OS is actively managing at a given moment and allows the user to monitor system health, troubleshoot bottlenecks, and terminate

unresponsive applications. In Windows, the Processes tab groups processes into applications and supporting services. The screenshot, see Figure 3, showed application processes Snipping Tool, Task Manager, and other services together with columns for CPU, memory, disk, and network components. This layout is helpful as it provides a snapshot of overall resource usage and application performance without necessarily requiring the user to understand command syntax. On the other hand, Linux used the command lines ``ps -eo pid,ppid,user,stat,%cpu,%mem,comm --sort=-%cpu | head -15`` to provide a process list ordered by `CPU` use; see Figure 2. The output shows `PID`, `PPID`, `USER`, `STAT`, `%CPU`, `%MEM`, and `COMMAND`, and application processes such as chrome, code, gnome-text-edit, and gnome-terminal. This means the Linux view emphasized structural process information instead of grouping entries by user-facing category. Guo et al. (2024) explain that process state and control depend on accurate tracking of process information, and the Linux listing made that internal structure much more explicit. The comparison shows a clear tradeoff. Windows is easier for quick human scanning, while Linux gives the user more immediate access to the metadata that an OS actually uses to distinguish one process from another.

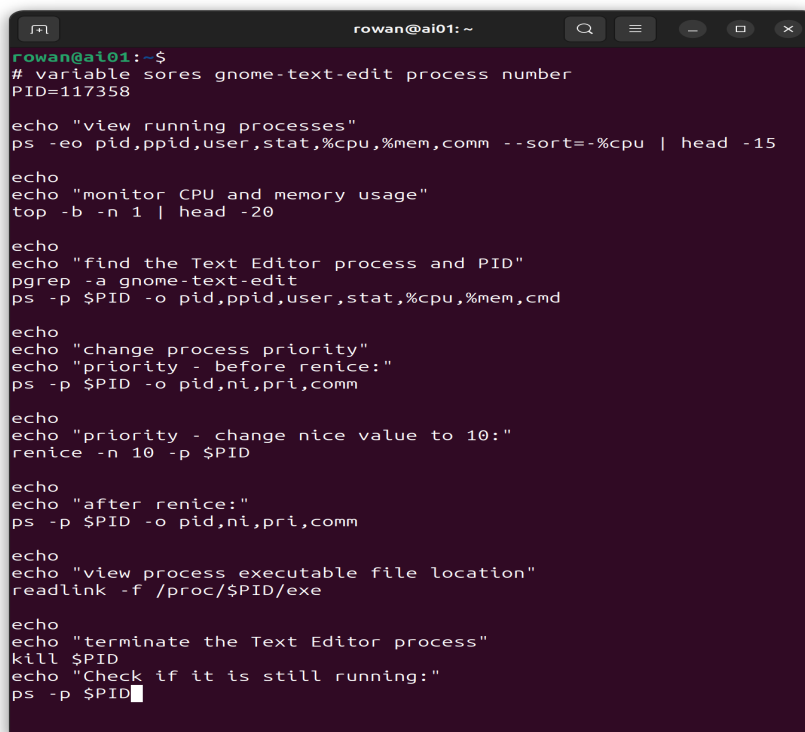
CPU and Memory Use

This task shows how process management can be used to monitor system load. In Windows, the Performance tab showed 10% CPU utilization, 74% memory use, 246 processes, 3365 threads, and 125651 handles; see Figure 4. The window screen also shows 4 cores and 8 logical processors are used. This adds detail that can be useful for the interpretation of the system's processing capabilities, as it may suggest that the processor was using simultaneous multithreading, showing more execution contexts to the scheduler. Note that this does not double the performance of the cores, but it does help the OS's ability to manage many runnable threads moving through the CPU at the same time. On the other hand, Linux used ``top -b -n 1 | head -`

20`, which generates a text-based rather than a graphical output. The Ubuntu output showed 548 total tasks, 1 running task, 547 sleeping tasks, 128488.9 MiB of total memory, and 8192.0 MiB of swap, with the highest-consumption processes listed below; see Figure 2 and 6. The processor was mostly idle at the moment the screenshot was taken. This shows that a large process count does not automatically mean heavy CPU usage. Moreover, this shows that the command `top` is useful not only because it lists active processes, but also because it shows whether the system is active and stable or it is overloaded. The Windows and Ubuntu commands provide the same kind of information but in different ways. Windows focuses on providing visual summaries or state, and Linux focuses on providing operational detail that can be copied, logged, or reused.

Figure 6

Ubuntu terminal with the pasted command sequence



```

rowan@ai01: ~
rowan@ai01:~$
# variable sores gnome-text-edit process number
PID=117358

echo "view running processes"
ps -eo pid,ppid,user,stat,%cpu,%mem,comm --sort=-%cpu | head -15

echo
echo "monitor CPU and memory usage"
top -b -n 1 | head -20

echo
echo "find the Text Editor process and PID"
pgrep -a gnome-text-edit
ps -p $PID -o pid,ppid,user,stat,%cpu,%mem,cmd

echo
echo "change process priority"
echo "priority - before renice:"
ps -p $PID -o pid,ni,pri,comm

echo
echo "priority - change nice value to 10:"
renice -n 10 -p $PID

echo
echo "after renice:"
ps -p $PID -o pid,ni,pri,comm

echo
echo "view process executable file location"
readlink -f /proc/$PID/exe

echo
echo "terminate the Text Editor process"
kill $PID
echo "Check if it is still running:"
ps -p $PID

```

Note: The screenshot shows the Linux commands prepared in the terminal before the output in Figure 2 was generated.

Finding a Specific Process and Its PID

Finding a process ID is often a critical first step, as many process-management tasks depend on identifying the correct process. In Windows, the Details tab displayed the PID column directly next to the process name. This makes identifying processes easier by keeping it in the same visual table as the process status and user name; see Figure 5. Linux used `pgrep -a gnome-text-edit` followed by `ps -p $PID -o pid,ppid,user,stat,%cpu,%mem,cmd`. The terminal output shows the Text Editor process as PID 117358, which can be expanded to show more detailed information; see Figure 2. This information or record showed the (application) owner, state, CPU share, memory share, and full command path. Guo et al. (2024) explain that process information is recorded and used to manage execution correctly. The Linux sequence implemented the same idea, but more clearly, it linked name matching to a structured process description in two direct steps. In other words, Windows implementation is simpler for a casual user, but Linux implementation makes the selection process more explicit. The user could see not only which process was selected, but also why it was the right one to use in later commands such as `renice`, `readlink`, or `kill`.

Terminating a Process

Terminating a process is one of the clearest forms of user intervention because it changes the active process set immediately. In Windows, Task Manager provides End task functionality through the Details tab context menu; see Figure 7. The command was easy to find and required no knowledge of signals, process states, or shell syntax. That is a strong usability advantage for local desktop troubleshooting. Linux used `kill $PID` after the Text Editor process had already been identified. The follow-up `ps -p $PID` output displayed only the header row and no active entry, which showed that the process was no longer running. Kerrisk (n.d.-a) explains that `kill` sends signals rather than merely forcing a window to close. This adds value to the Linux

Figure 8

Windows view after the termination workflow

Name	PID	Status	User name	CPU	Work...	Platform	UAC virtualiza...
AggregatorHost.exe	7308	Running	SYSTEM	00	0 K	64 bit	Not allowed
AppHelperCap.exe	3452	Running	SYSTEM	00	0 K	64 bit	Not allowed
ApplicationFrame...	2112	Running	copya	00	0 K	64 bit	Disabled
AppVShNotify.exe	1564	Running	SYSTEM	00	0 K	64 bit	Not allowed
audiodg.exe	6548	Running	LOCAL SE...	00	0 K	64 bit	Not allowed
backgroundTaskH...	13968	Suspended	copya	00	-24 K	64 bit	Disabled
backgroundTaskH...	8988	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	11696	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	16304	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	8700	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	3936	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	14840	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	11796	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	11252	Suspended	copya	00	0 K	64 bit	Disabled
backgroundTaskH...	18756	Suspended	copya	00	0 K	64 bit	Disabled
BridgeCommunica...	1960	Running	copya	00	0 K	64 bit	Disabled
conhost.exe	9924	Running	copya	00	0 K	64 bit	Disabled
CrossDeviceResum...	7076	Running	copya	00	0 K	64 bit	Disabled
CrossDeviceService...	10704	Running	copya	00	-8 K	64 bit	Disabled
csrss.exe	956	Running	SYSTEM	00	0 K	64 bit	Not allowed
csrss.exe	700	Running	SYSTEM	00	0 K	64 bit	Not allowed
ctfmon.exe	15156	Running	copya	00	0 K	64 bit	Disabled
dasHost.exe	7744	Running	LOCAL SE...	00	0 K	64 bit	Not allowed
dasHost.exe	6064	Running	NETWORK...	00	0 K	64 bit	Not allowed
DataExchangeHost...	16956	Running	copya	00	0 K	64 bit	Disabled
DesktopExtension.e...	19072	Running	copya	00	4 K	64 bit	Disabled
DiagsCap.exe	3468	Running	SYSTEM	00	0 K	64 bit	Not allowed

Note: The screenshot shows the Details view after the termination action returned the user to the process list.

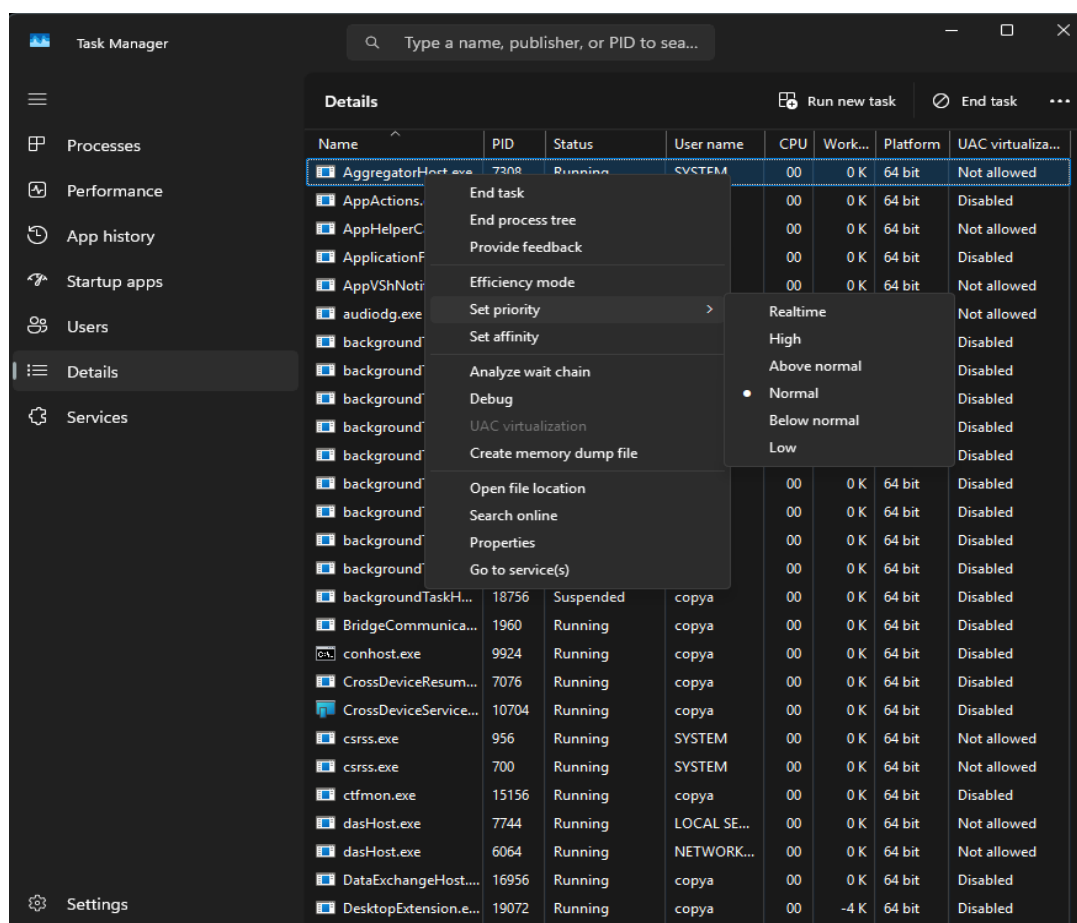
Changing Process Priority

Changing process priority shows how process management affects scheduling rather than simple observation. In Windows, the Details tab exposed a Set priority menu with levels such as 'Realtime', 'High', 'Above normal', 'Normal', 'Below normal', and 'Low'; see Figures 9 and 10. The menu is easy to understand because the choices are expressed in familiar language instead of numeric values. Linux used `ps -p $PID -o pid,ni,pri,comm` before and after `renice -n 10 -p $PID`. The output showed that the test process moved from niceness 0 and priority 19 to niceness 10 and priority 9; see Figure 2. This means the scheduler treated that process less

favorably after the change, which is useful when a user wants a task to consume fewer scheduling opportunities relative to other work. Guo et al. (2024) connect scheduling to process control, and the Linux example makes that connection easy to observe because the changed scheduling attributes are shown directly in the output. Windows made the action more approachable, but Linux made the scheduling consequences more transparent. For an operating-systems course, that transparency is especially useful because it connects the user action to the scheduler's internal decision-making process.

Figure 9

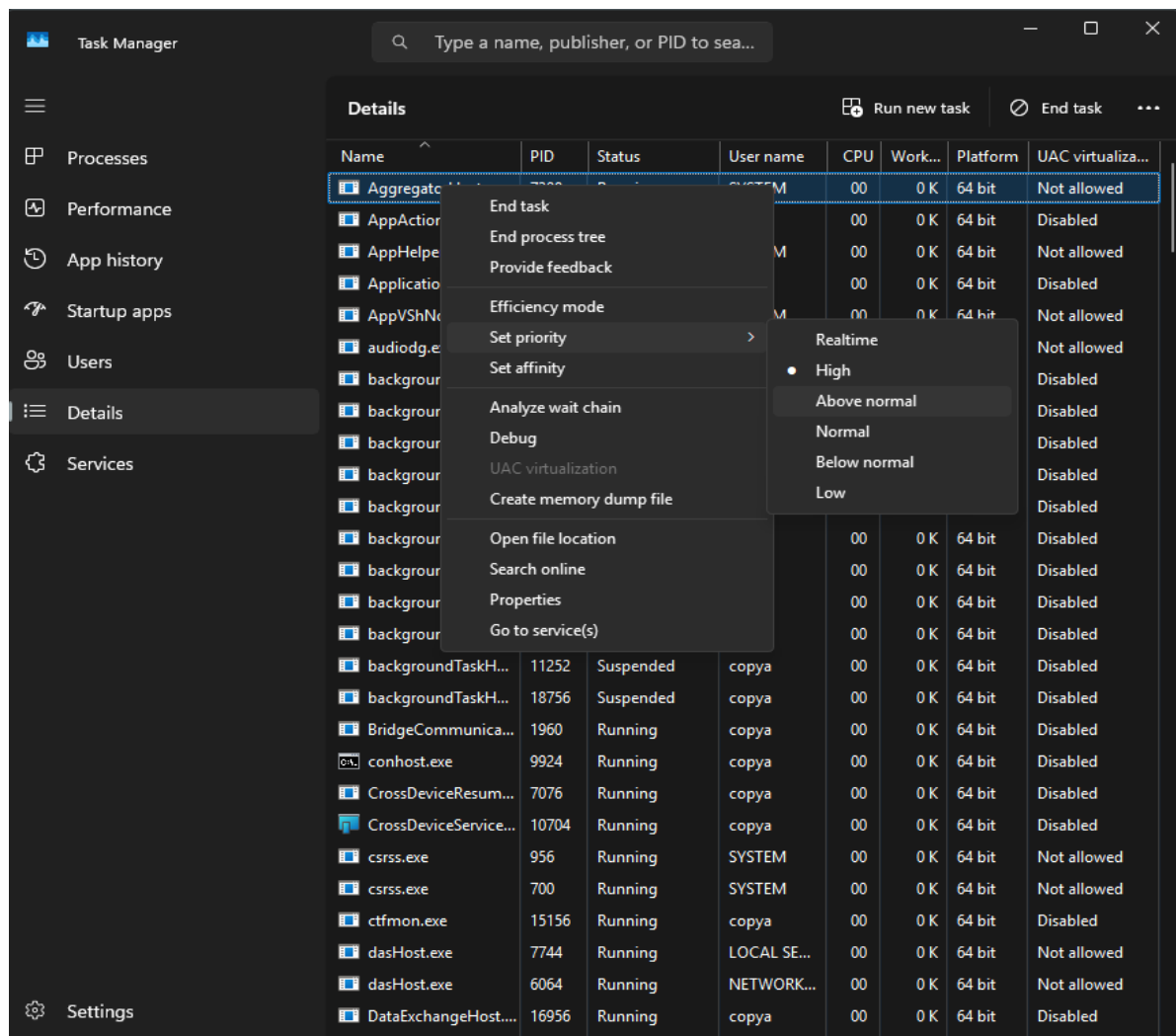
Windows priority menu before the priority change



Note: The screenshot shows the full list of Windows priority classes available through Task Manager.

Figure 10

Windows priority menu highlighting a new selection



Note: The screenshot shows above normal highlighted in the Task Manager priority submenu.

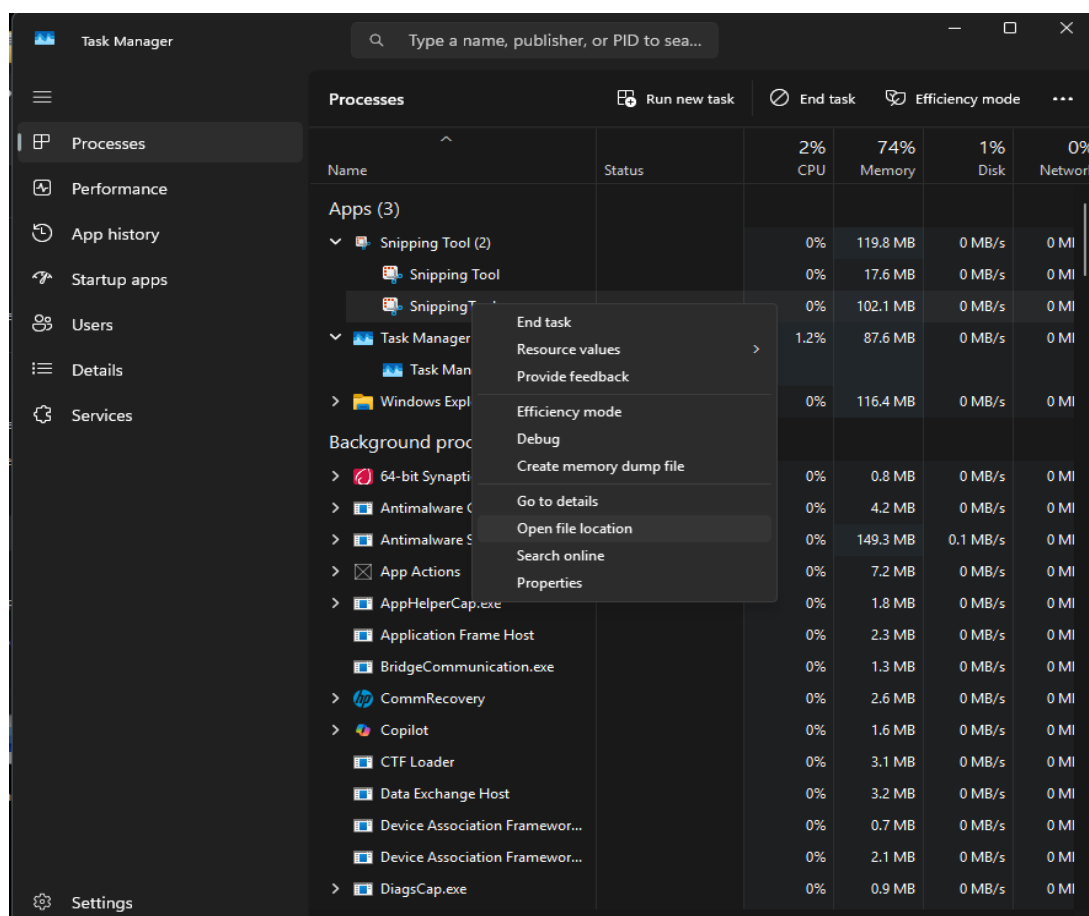
Viewing the Executable File Location

Locating the executable file behind a running process adds a useful file-system dimension to process management. In Windows, Task Manager supported this through Open file location, see Figures 11 and 12, which launched File Explorer and opened the directory containing the selected executable. That feature is practical because it helps the user connect a running process to the actual program stored on disk during troubleshooting or verification. Linux used `readlink`

`-f /proc/$PID/exe`, which resolved the executable path to `/usr/bin/gnome-text-editor`; see Figure 2. Kerrisk (n.d.-c) explains that `/proc/<PID>/exe` points to the pathname of the executed command. Therefore, Linux reached the same kind of answer through a more direct operating-system interface. Instead of opening a graphical folder view first, it resolved the executable path from the kernel's own process representation. The Windows method is more convenient for users who want an immediate file browser result. The Linux method is more exact and more scriptable. Both are useful, but they reflect different design priorities.

Figure 11

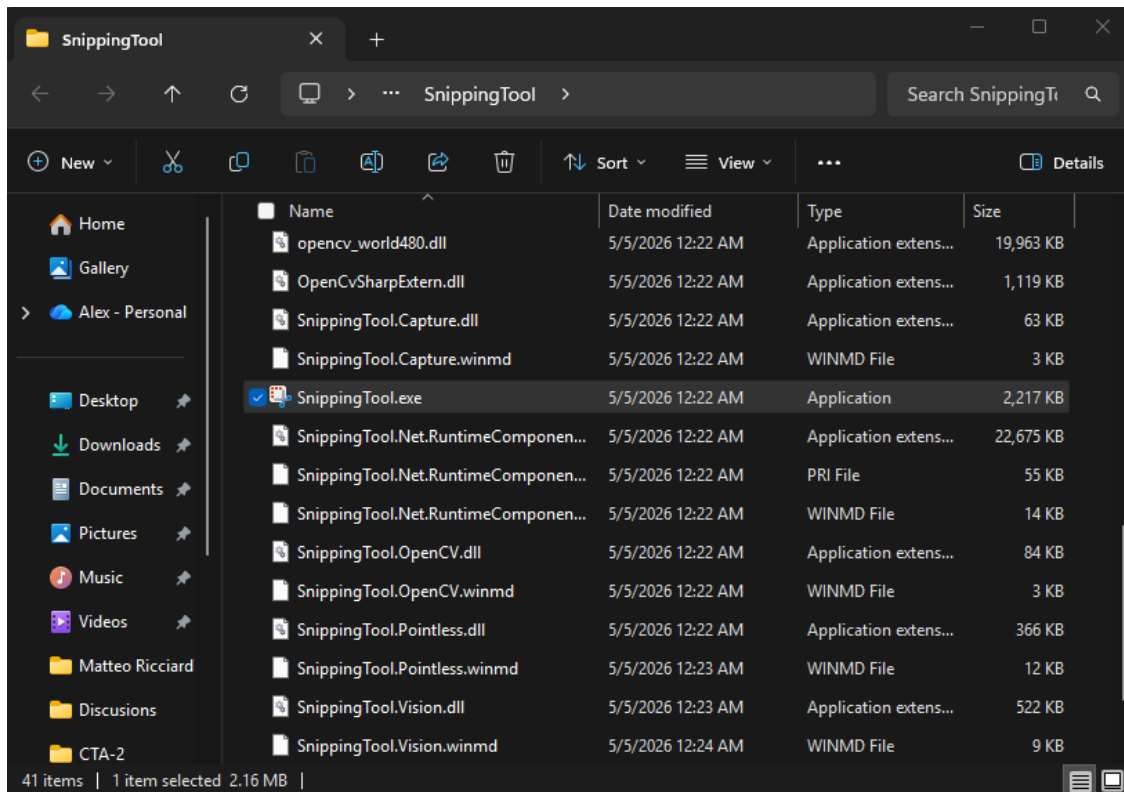
Windows Open file location command from Task Manager



Note: The screenshot shows the command used to move from a running Windows process to its executable location.

Figure 12

Windows File Explorer view of the executable location



Note: The screenshot shows the resulting File Explorer view of the SnippingTool executable directory.

Ease of Use and Ability to Intervene

Taken together, the six tasks show that both OSs support meaningful process observation and intervention. Windows Task Manager was easier to use because the graphical layout reduced the amount of prior knowledge needed to inspect or control a process. A user could move from viewing processes to ending a task or changing priority through menus and context actions alone. Linux shell commands were less immediately approachable, but they provided stronger low-level visibility and repeatability. For example, a set of commands can be reused in a script, remote session, or technical report without changing its basic logic. This is very useful as it allows for

reproducing process-management steps and processes, and it allows for documenting and verifying the system against the system's current state. The two systems, therefore, differ less in capability than in emphasis. Windows favors ease of use and convenience. Linux emphasizes precision, explicit state visibility, and command-level control. For learning OS concepts, Linux makes more of the mechanisms visible. For quick intervention on a local desktop, Windows is often easier. Saying that Windows 11 also provides command-line process management commands through PowerShell, such as `'Get-Process'` and `'Stop-Process'`, therefore, Windows 11 is not limited to graphical process management through Task Manager and provides similar command-line process management methods to Linux.

Conclusion

This comparison shows that Windows Task Manager and Ubuntu/Linux shell utilities can both perform the core process-management tasks. Windows provided a graphical path for viewing processes, monitoring resource use, identifying PIDs, terminating tasks, changing priority, and opening an executable's file location. Linux performed the same general functions through `'ps'`, `'top'`, `'pgrep'`, `'ps -p'`, `'renice'`, `'readlink'`, and `'kill'`. Nonetheless, Windows 11 also provides process management command-line capability, as PowerShell can provide similar process-management control through commands such as `'Get-Process'` and `'Stop-Process'`. Finally, what this comparison analysis shows is that process management is not only about listing what is running. It is also about understanding process state, scheduling priority, executable identity, and controlled intervention. Windows Task Manager presented those concepts in a simpler and visual form, while the Linux terminal shows them in a more technically explicit form.

References

- Bazuku, R., Anab, A., Gyemerah, S., & Daabo, M. I. (2023). *An overview of computer OSs and emerging trends. Asian Journal of Research in Computer Science*, 16(4), 161-177.
<https://doi.org/10.9734/AJRCOS/2023/v16i4380>
- Fisher, T. (n.d.). *Task manager: What it is and how to use it*. Lifewire.
<https://www.lifewire.com/task-manager-2626025>
- Guo, J., Wu, S., Chen, H., Huang, S., & Zhang, Y. (2024). Application analysis of process state control in modern OS. *Academic Journal of Computing & Information Science*, 7(6), 13-19. <https://doi.org/10.25236/AJCIS.2024.070603>
- Kerrisk, M. (n.d.-a). *kill(1) - Linux manual page*. man7.org. <https://man7.org/linux/man-pages/man1/kill.1.html>
- Kerrisk, M. (n.d.-b). *pgrep(1) - Linux manual page*. man7.org. <https://man7.org/linux/man-pages/man1/pgrep.1.html>
- Kerrisk, M. (n.d.-c). *proc_pid_exe(5) - Linux manual page*. man7.org.
https://man7.org/linux/man-pages/man5/proc_pid_exe.5.html
- Kerrisk, M. (n.d.-d). *ps(1) - Linux manual page*. man7.org. <https://man7.org/linux/man-pages/man1/ps.1.html>
- Kerrisk, M. (n.d.-e). *renice(1) - Linux manual page*. man7.org. <https://man7.org/linux/man-pages/man1/renice.1.html>
- Kerrisk, M. (n.d.-f). *top(1) - Linux manual page*. man7.org. <https://man7.org/linux/man-pages/man1/top.1.html>

Microsoft Learn. (2026, February 12). *Troubleshoot processes by using Task Manager*.

Microsoft. <https://learn.microsoft.com/en-us/troubleshoot/windows-server/support-tools/support-tools-task-manager>

Microsoft Support. (n.d.). *Tips to improve PC performance in Windows*. Microsoft.

<https://support.microsoft.com/en-us/windows/tips-to-improve-pc-performance-in-windows-b3b3ef5b-5953-fb6a-2528-4bbcd82fba96>

Rawat, V. (2025). *A brief study on OSs along with their history, types, working, challenges and advancements* [Preprint]. Zenodo. <https://doi.org/10.5281/zenodo.15869774>